Whole System Design

System Design Document: Hospitality Management System

1. System Design Document

High-Level Architecture

For the MVP, we will use a Modular Monolith architecture. This provides the fast development speed needed for an MVP while maintaining clear boundaries between domains.

Component Breakdown

- **Authentication & Authorization Module:** Handles user registration, login, and session management using JSON Web Tokens (JWT). It enforces Role-Based Access Control (RBAC) with three primary roles: Guest, Staff, and Admin.
- **Booking Module:** Manages the core reservation logic. It checks availability, calculates costs, creates reservation records, and handles cancellations.
- **Room Management Module:** Maintains the source of truth for physical rooms. It tracks room types, features, and real-time statuses (e.g., vacant, occupied, cleaning). It also integrates with the access control system (e.g., generating digital door PINs).
- **Guest Services Module:** A ticketing system for handling guest requests during their stay. It tracks the lifecycle of requests (e.g., extra towels, room service, maintenance) and dispatches notifications to staff.

Database Design

We will use a relational database (PostgreSQL)

Key Entities & Relationships:

- **User**: (1 to Many) -> Reservation, ServiceRequest
- **Room**: (1 to Many) -> Reservation
- **Reservation**: Links a User and a Room for a specific timeframe.

Brief Schema Description (MVP):

- **Users:** id, name, email, password_hash, role (Stores guest and staff profiles)
- **Rooms:** id, room_number, type, size_sqm, max_guests, status, base_price (Tracks physical rooms and current states), created_at
- **Amenities**: id, name("Mountain View", "Trouser Press", "WiFi"), category('View', 'Electronic', 'Comfort', 'Bathroom'), icon_slug("security", “wifi”)
- **Room Amenities**: room_id, amenity_id - this is a junction table.
- **Reservations:** id, user_id, room_id, check_in_date, check_out_date, status

- **Service_Requests:** id, reservation_id, request_type, status, created_at (Logs guest requests linked to their current stay)

API Design

The system will expose RESTful APIs. Here are the main endpoints for the MVP:

- **Authentication**
  - POST /api/auth/register: Register a new guest.
  - POST /api/auth/login: Authenticate and return a JWT.
- **Booking**
  - GET /api/bookings/availability: Check available rooms for given date ranges.
  - POST /api/bookings: Create a new reservation.
  - GET /api/bookings/{id}: Retrieve reservation details.
- **Room Management**
  - GET /api/rooms: List all rooms (Staff/Admin only).
  - PUT /api/rooms/{id}/status: Update room status (e.g., to 'Needs Cleaning').
- **Guest Services**
  - POST /api/services/requests: Submit a new guest request.
  - GET /api/services/requests: List pending requests (Staff only).
  - PUT /api/services/requests/{id}/status: Resolve or update a request.
-

2. Detailed Design Considerations

Access Reservation Workflows

End-to-end flow from booking to access validation.

1. **Search:** Guest inputs dates. The Booking module queries the database for available rooms and returns options.
2. **Booking:** Guest selects a room and confirms. The system creates a Reservation record with status Confirmed.
3. **Access :** Upon successful booking, the Room Management module sends a confirmation email to a customer and attaches it to the reservation.
4. **Check-in Day:** Guest receives an email/notification with the details of booking.
5. **Access Validation:** Guests enter the room taking the key from the front desk.
6. **State Change:** Upon first entry, the reservation status updates to Checked-In, and the room status updates to Occupied.

Room Lifecycle Management

Managing availability, cleaning, and maintenance states.

The room lifecycle is governed by a strict State Machine to prevent double-booking and ensure hygiene.

- **Available:** Ready to be booked. Clean and functional.
- **Booked:** Reserved for future dates, but currently empty.
- **Occupied:** Guest has checked in.
- **Needs Cleaning:** Triggered automatically when a guest checks out (reservation ends). Room cannot be booked for immediate check-in.
- **In Maintenance:** Triggered manually by staff if an issue is reported (e.g., broken AC). Overrides other states until resolved.
- **Workflow:** Occupied -> (Guest Checkout) -> Needs Cleaning -> (Housekeeping updates app) -> Available.

Guest Service Flows

Handling requests, support, and notifications.

1. **Submission:** Guest uses the portal to request a service (e.g., "Late Checkout", "Extra Towels"). An entry is created in Service_Requests with status Pending.
2. **Notification:** The system triggers a real-time notification (via WebSockets or Server-Sent Events) to the Staff Dashboard.
3. **Acknowledgment:** A staff member claims the ticket. Status changes to In Progress.
4. **Resolution:** Once the task is completed, staff marks it Resolved.
5. **Feedback Loop:** The system sends an automated notification (email or app push) back to the guest confirming the request has been fulfilled.

Booking Module

Booking Module — Detailed Design Document

1. Overview

The Booking Module is the core transactional engine of the platform. It owns the full reservation lifecycle — from a guest searching for available rooms, through confirmation, modification, and cancellation, to the final checkout handoff. It is the single source of truth for what room is reserved by whom and when.

Every other module interacts with the Booking Module: Room Management reads booking state to update room status, the Experience & Services Module links service bookings to a reservation, and Billing accumulates charges against a reservation ID.

2. Responsibilities

- Check room availability for a given date range, location, and guest count
- Create, confirm, and manage reservations
- Enforce cancellation and modification policies
- Trigger downstream actions (room status update, billing initialisation)
- Expose reservation data to guests, staff, and admin with appropriate access controls

3. Data Model

3.1 Core Tables

Reservations

id	UUID	Primary key
user_id	UUID	FK → Users
room_id	UUID	FK → Rooms

resort_id	UUID	FK → Resorts
check_in_date	DATE	Required
check_out_date	DATE	Required
adults	INT	Default 1
children	INT	Default 0
status	ENUM	confirmed | checked_in | checked_out | cancelled | no_show
base_price_cents	INT	Nightly rate at time of booking (snapshot)
total_price_cents	INT	Calculated total at confirmation
special_requests	TEXT	Free-text guest notes
cancellation_reason	TEXT	Populated on cancellation
cancelled_at	TIMESTAMP	Nullable
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

— Index on (room_id, check_in_date, check_out_date)

— Index on (user_id, status)

— Index on (resort_id, check_in_date)

Reservation_Audit_Log

id	UUID	
reservation_id	UUID	FK → Reservations
changed_by	UUID	FK → Users
previous_status	ENUM	
new_status	ENUM	
change_reason	TEXT	
changed_at	TIMESTAMP	

The audit log is append-only. Every status change to a reservation writes a new row. This supports dispute resolution and guest service history.

3.2 Key Relationships

Users ——(1:many)——▸ Reservations

Rooms ——(1:many)——▸ Reservations

Resorts ——(1:many)——▸ Reservations

Reservations ——(1:many)——▸ ServiceBookings

Reservations ——(1:many)——▸ ServiceRequests

Reservations ——(1:many)——▸ Payments

4. Availability Logic

4.1 Query

Availability is checked by querying rooms that do NOT have a conflicting active reservation for the requested date range.

```
SELECT r.*

FROM rooms r

WHERE r.resort_id = :resort_id

  AND r.max_guests >= :guest_count

  AND r.status != 'in_maintenance'

  AND r.id NOT IN (

    SELECT room_id FROM reservations

    WHERE status IN ('confirmed', 'checked_in')

      AND check_in_date  < :check_out_date

      AND check_out_date > :check_in_date

  )
```

4.2 Caching

Availability results are cached in Redis with a 60-second TTL. Cache is invalidated immediately on any reservation creation or cancellation for that resort and date range.

4.3 Race Condition Protection

When a guest moves from search to booking confirmation, the system uses a PostgreSQL `SELECT FOR UPDATE` lock on the target room to prevent two guests confirming the same room simultaneously. If the lock cannot be acquired within 3 seconds, the system returns a `409 Conflict` and prompts the guest to re-search.

5. Pricing Calculation

Price is calculated at the time of booking and snapshotted onto the reservation. It is not recalculated after confirmation unless the reservation is modified.

```
total_price = base_nightly_rate
            × number_of_nights
            + applicable_taxes
            + applicable_fees (resort fee, cleaning fee, etc.)
            - applicable_discounts (promo codes, loyalty tier)
```

The `base_price_cents` on the reservation stores the nightly rate at the time of booking. This protects against price changes affecting confirmed reservations.

Post-MVP: a dynamic pricing engine can plug in here to adjust `base_nightly_rate` based on occupancy, seasonality, and lead time.

---

6. Reservation Lifecycle

6.1 Status Flow

```
[confirmed] ──────────────────────────────────────────────────────► [checked_in]
    │                                   │
    │ (guest cancels or no-show)        │ (checkout)
    ▼                                   ▼
[cancelled]                         [checked_out]
```

[no_show]

6.2 Status Definitions

Status	Description
`confirmed`	Reservation created and payment authorised. Room is blocked.
`checked_in`	Guest has entered the room.
`checked_out`	Stay completed. Invoice finalised. Room released for cleaning.
`cancelled`	Reservation cancelled before check-in. Refund policy applied.
`no_show`	Guest did not arrive by end of check-in window. Charged per policy.

7. Cancellation & Modification Policy

7.1 Cancellation Rules

Time before check-in	Refund

≥ 72 hours	Full refund
< 72 hours	No refund

Cancellation policy is enforced in the module at the time of cancellation. The applicable policy is determined by the `check_in_date` on the reservation and the current timestamp.

7.2 Modifications

Guests can modify dates or room type if the target room is available. Modifications are treated as a cancellation + rebook internally, with the price difference settled at the time of change. If a modification would trigger a cancellation fee, the guest is warned before confirming.

Modifications are only allowed if `reservation.status = 'confirmed'`. A reservation that is already `checked_in` cannot be modified.

9. API Endpoints

9.1 Endpoint Reference

GET /api/bookings/availability

POST /api/bookings

GET /api/bookings

GET /api/bookings/:id

PATCH /api/bookings/:id

DELETE /api/bookings/:id

POST /api/bookings/:id/checkin

POST /api/bookings/:id/checkout

9.2 Endpoint Details

`GET /api/bookings/availability`

Returns available rooms for the given search criteria.

Query parameters:

Param	Type	Required	Description
`resort_id`	UUID	Yes	Target resort
`check_in`	DATE	Yes	Format: YYYY-MM-DD
`check_out`	DATE	Yes	Format: YYYY-MM-DD
`guests`	INT	Yes	Total guest count
`type`	STRING	No	Filter by room type

Response `200 OK`:

```
{
  "available_rooms": [
```

```
    {
      "id": "uuid",
      "room_number": "201",
      "type": "deluxe_double",
      "size_sqm": 42,
      "max_guests": 2,
      "base_price_cents": 18000,
      "amenities": ["WiFi", "Lake View", "Fireplace"],
      "images": ["url1", "url2"]
    }
  ],
  "currency": "USD",
  "nights": 3
}
```


`POST /api/bookings`

Creates a new reservation.

Request body:


```
{
  "room_id": "uuid",
  "check_in_date": "2025-11-01",
```

```
  "check_out_date": "2025-11-04",

  "adults": 2,

  "children": 0,

  "special_requests": "High floor preferred"

}
```

Behaviour:

1. Validates date range (check-out must be after check-in, minimum 1 night).
2. Re-checks availability with `SELECT FOR UPDATE`.
3. Calculates total price including taxes.
4. Creates reservation with status `confirmed`.
5. Triggers: Room status → `booked`, billing initialisation.

Response `201 Created`:

```
{

  "reservation_id": "uuid",

  "status": "confirmed",

  "check_in_date": "2025-11-01",

  "check_out_date": "2025-11-04",

  "total_price_cents": 57600,
```

```
{
  "reservation_id": "uuid",
  "status": "confirmed",
  "check_in_date": "2025-11-01",
  "check_out_date": "2025-11-04",
  "total_price_cents": 57600
}
```

Error responses:

Code	Reason
`400`	Invalid date range or missing fields
`409`	Room no longer available (race condition)
`422`	Business rule violation (e.g. max guests exceeded)

`GET /api/bookings/:id`

Returns full reservation detail.

Access control: Guest can only retrieve their own reservations. Staff and above can retrieve any.

Response `200 OK`:


```
{
  "id": "uuid",
  "room": { "id": "uuid", "room_number": "201", "type": "deluxe_double" },
  "resort": { "id": "uuid", "name": "Lakeside Resort" },
  "check_in_date": "2025-11-01",
  "check_out_date": "2025-11-04",
  "status": "confirmed",
  "adults": 2,
  "children": 0,
  "base_price_cents": 18000,
  "total_price_cents": 57600,
  "special_requests": "High floor preferred",
  "service_bookings": [],
  "created_at": "2025-10-15T09:30:00Z"
}
```


`DELETE /api/bookings/:id`

Cancels a reservation. Only allowed if `status = 'confirmed'`.

Response `200 OK`:

{

"reservation_id": "uuid",

"status": "cancelled",

"refund_amount_cents": 57600,

"refund_policy": "full_refund",

"message": "Cancellation confirmed. Refund will be processed within 5–7 business days."

}

`POST /api/bookings/:id/checkin`

Marks a reservation as checked in. Triggered automatically on first entry or manually by staff.

Behaviour:

1. Validates `check_in_date` is today or in the past.
2. Updates `reservation.status` → `checked_in`.
3. Updates `room.status` → `occupied`.
4. Writes to audit log.

`POST /api/bookings/:id/checkout`

Initiates checkout. Only allowed if `status = 'checked_in'`.

Behaviour:

1. Validates all outstanding service bookings are `completed` or `cancelled`.
2. Flags any unpaid charges.
3. Updates `reservation.status` → `checked_out`.
4. Updates `room.status` → `needs_cleaning`.
5. Triggers invoice generation.

10. Notifications

Event	Channel	Recipient
Reservation confirmed	Email + Push	Guest
Check-in reminder (24h before)	Email + Push	Guest
Checkout reminder (morning of)	Push	Guest
Reservation cancelled	Email	Guest
New reservation created	Dashboard alert	Staff / Manager
No-show flagged	Dashboard alert + Email	Manager

11. Business Rules Summary

- A reservation cannot span more than 90 nights (configurable per resort).
- Check-in time is 14:00; checkout is 12:00 (configurable per resort).
- A guest cannot have more than 5 active (confirmed) reservations across all resorts simultaneously.
- Reservations for past dates cannot be created.
- Rooms in `in_maintenance` status cannot be booked.
- Only Admin can override a cancellation fee waiver.

- Price is always calculated in the resort's base currency and stored in cents to avoid floating-point issues.

Room Management Module

Room Management Module — Detailed Design Document

1. Overview

The Room Management Module is the source of truth for every physical room across all resort properties. It owns room configuration, real-time status tracking, inventory management, and amenity cataloguing. Where the Booking Module owns the reservation, this module owns the room itself.

It operates as a state machine: every room is always in one of five defined states, and transitions between states are strictly enforced. This prevents double-booking, ensures cleaning standards are met before a room is re-let, and gives management real-time visibility across the entire property portfolio.

2. Responsibilities

- Define and maintain room inventory per resort location
- Track room types, configurations, sizes, and features
- Enforce the room status state machine and all transition rules
- Manage the amenity catalog and room-amenity assignments
- Provide real-time room availability and status data to the Booking and Staff modules
- Support housekeeping and maintenance workflows
- Expose room data to guests (browsable), staff (operational), and admin (full control)

3. Data Model

3.1 Core Tables

```
Rooms
  id              UUID        Primary key
  resort_id       UUID        FK → Resorts
  room_number     VARCHAR     Unique per resort (e.g. "201", "Suite A")
  type            ENUM        standard | superior | deluxe | suite | presidential
  floor           INT
  size_sqm        DECIMAL
  max_guests      INT
```

```
  bed_configuration   VARCHAR       (e.g. "1 King", "2 Queen", "Twin")
  status              ENUM          available | booked | occupied | needs_cleaning | in_maintenance
  base_price    INT           Nightly rack rate
  accessible          BOOLEAN       Wheelchair / accessibility compliant
  notes               TEXT          Internal staff notes
  created_at          TIMESTAMP
  updated_at          TIMESTAMP

  — Unique index on (resort_id, room_number)
  — Index on (resort_id, status)
  — Index on (resort_id, type, status) for availability filtering

Amenities
  id                  UUID          Primary key
  name                VARCHAR       e.g. "Lake View", "WiFi", "Fireplace", "Jacuzzi"
  category            ENUM          view | technology | comfort | bathroom | outdoor
  icon_slug           VARCHAR       e.g. "wifi", "fire", "mountain-view"
  created_at          TIMESTAMP

Room_Amenities  ← junction table
  room_id             UUID          FK → Rooms
  amenity_id          UUID          FK → Amenities
  PRIMARY KEY (room_id, amenity_id)

Room_Status_Log
  id                  UUID
  room_id             UUID          FK → Rooms
  previous_status     ENUM
  new_status          ENUM
  changed_by          UUID          FK → Users (staff/system)
  reason              TEXT          e.g. "AC unit broken", "Guest checked out"
  changed_at          TIMESTAMP

  — Append-only. Never deleted.
  — Index on (room_id, changed_at)

Room_Images
  id                  UUID
  room_id             UUID          FK → Rooms
  url                 VARCHAR
  caption             VARCHAR
  sort_order          INT
  created_at          TIMESTAMP
```

3.2 Key Relationships

```
Resorts    ──(1:many)──▸ Rooms
Rooms      ──(1:many)──▸ Reservations
Rooms      ──(many:many)─▸ Amenities (via Room_Amenities)
Rooms      ──(1:many)──▸ Room_Status_Log
Rooms      ──(1:many)──▸ Room_Images
```

4. Room Status State Machine

All status transitions are validated server-side. A `PATCH /api/rooms/:id/status` request that attempts an illegal transition returns `HTTP 422 Unprocessable Entity`.

4.1 Status Definitions

Status	Description
`available`	Room is clean, functional, and ready to be booked or walked into.
`booked`	A confirmed reservation exists for future dates. Room is empty today.
`occupied`	Guest has checked in.
`needs_cleaning`	Guest has checked out or housekeeping is required. Not available for new check-ins until cleared.
`in_maintenance`	A fault has been reported or maintenance is in progress. Blocks all new bookings.

4.2 Allowed Transitions

```
available    ──▸ booked          (Booking Module: reservation confirmed)
booked       ──▸ available       (Booking Module: reservation cancelled)
booked       ──▸ occupied        (Booking Module: guest checks in)
booked       ──▸ needs_cleaning  (Housekeeping requested before guest arrival)
booked       ──▸ in_maintenance  (Staff reports fault)
occupied     ──▸ needs_cleaning  (Booking Module: guest checks out — automatic)
```

needs_cleaning ⟶ available (Housekeeping: room cleared and inspected)
needs_cleaning ⟶ in_maintenance (Fault found during cleaning)
in_maintenance ⟶ available (Manager: issue resolved)
in_maintenance ⟶ needs_cleaning (Maintenance complete; cleaning required before re-letting)

Any other transition (e.g. `available → occupied`, skipping `booked`) is rejected.

4.3 Transition Rules

- Only the system (via Booking Module) can set a room to `booked` or `occupied`.
- Only Staff or above can set a room to `needs_cleaning` or `in_maintenance`.
- Only Manager or Admin can clear `in_maintenance` back to `available` or `needs_cleaning`.
- Every transition writes a row to `Room_Status_Log` with the actor and reason.
- A room in `in_maintenance` cannot receive new reservations until a Manager explicitly clears it.

4.4 Automatic Transitions

Trigger	From	To	Actor
Reservation confirmed	`available`	`booked`	System (Booking Module)
Reservation cancelled	`booked`	`available`	System (Booking Module)
Guest checks in	`booked`	`occupied`	System (Booking Module)
Guest checks out	`occupied`	`needs_cleaning`	System (Booking Module)
Housekeeping marks complete	`needs_cleaning`	`available`	Staff (Housekeeping app)

5. Room Types & Configuration

5.1 Room Type Hierarchy

standard Entry-level room. Standard amenities, single or double bed.

superior	Upgraded furnishings, slightly larger, may have a view.
deluxe	Premium furnishings, lake or garden view, larger bathroom.
suite	Separate living area, full amenities, premium service tier.
presidential	Top-tier. Full suite with dedicated butler service. (1–2 per resort.)

Each type carries a base price multiplier applied on top of the resort's floor rate. Exact pricing is stored per room in `rooms.base_price`

5.2 Bed Configurations

Stored as a free-text field per room to allow flexibility. Standard values:

- `1 King`
- `1 Queen`
- `2 Queen`
- `Twin`
- `King + Sofa Bed`

5.3 Accessibility

Rooms flagged `accessible = true` appear in filtered availability results when a guest requests accessible accommodation. These rooms have wider doorways, roll-in showers, lowered fixtures, and emergency pull cords. Accessibility features are also represented as Amenities for UI display.

6. Amenity Catalog

Amenities are managed centrally and assigned to rooms via the `Room_Amenities` junction table. This allows a single amenity (e.g. "WiFi") to be assigned to many rooms, and a room to have many amenities.

6.1 Amenity Categories

Category	Example Amenities
`view`	Lake View, Garden View, Mountain View, City View, Pool View
`technology`	WiFi, Smart TV, Bluetooth Speaker, Work Desk, USB Ports
`comfort`	Fireplace, Air Conditioning, Blackout Curtains, Sofa, King Bed

`bathroom`	Jacuzzi, Rain Shower, Steam Room, Premium Toiletries, Bathrobe
`outdoor`	Private Balcony, Plunge Pool, Terrace, Direct Garden Access

6.2 Managing Amenities

- Amenities are created and managed by Admin only.
- Room-amenity assignments are managed by Admin or Manager.
- The `icon_slug` field maps to the front-end icon library for consistent display.
- Removing an amenity from a room does not delete the amenity record — it deletes the `Room_Amenities` row.

8. Housekeeping Workflow

8.1 Flow

1. Room transitions to [needs_cleaning]
   ↓
2. Housekeeping dashboard shows room in queue
   ↓
3. Housekeeper is assigned to the room (by Manager or self-assigns)
   ↓
4. Housekeeper marks room [in_progress] on mobile app
   ↓
5. Cleaning completed → housekeeper marks room [inspected]
   ↓
6. Supervisor optionally performs quality check
   ↓
7. Room marked [available] — ready for next guest

8.2 Priority Queue

Rooms appear in the housekeeping queue ordered by:

1. Imminent arrival (next reservation check-in within 4 hours — highest priority)
2. Time since checkout (longest waiting first)
3. Room type (presidential and suites prioritised over standard)

8.3 Notifications

Event	Channel	Recipient

Room enters `needs_cleaning`	Dashboard alert + Push	Housekeeping staff
High-priority room (imminent arrival)	Push	Housekeeping supervisor
Room marked `available`	Dashboard alert	Front desk / Booking system
Room in `needs_cleaning` for > 2 hours	Escalation alert	Manager

9. Maintenance Workflow

9.1 Flow

1. Fault reported — by guest (via Guest Services) or staff inspection
   ↓
2. Manager or Staff sets room status → [in_maintenance]
   ↓
3. Maintenance ticket created and assigned to maintenance staff
   ↓
4. Maintenance team works on room
   ↓
5. Maintenance marks work complete
   ↓
6. Manager inspects and resolves:
   → [available] (if no cleaning required)
   → [needs_cleaning] (if cleaning required before re-letting)

9.2 Impact on Reservations

When a room is set to `in_maintenance`:

- No new reservations can be created for that room.
- If an active reservation exists within the affected dates, the system alerts the Manager immediately.
- The Manager must manually relocate the affected guest (reassign to an equivalent available room) and notify them.
- Relocation logic is not automatic — it requires Manager judgment to match guest preferences.

10. API Endpoints

10.1 Endpoint Reference

GET /api/rooms List all rooms (Staff+)
GET /api/rooms/:id Get room detail
POST /api/rooms Create a room (Admin)
PUT /api/rooms/:id Update room configuration (Admin)
PATCH /api/rooms/:id/status Update room status (state machine enforced)
GET /api/rooms/:id/status-log Get full status history for a room
GET /api/rooms/:id/availability Check if specific room is available for dates

GET /api/amenities List all amenities
POST /api/amenities Create an amenity (Admin)
POST /api/rooms/:id/amenities Assign amenity to a room
DELETE /api/rooms/:id/amenities/:amenity_id Remove amenity from a room

GET /api/rooms/housekeeping/queue Get rooms needing cleaning (Staff+)
PATCH /api/rooms/:id/housekeeping Update housekeeping progress

10.2 Endpoint Details

`GET /api/rooms`

Returns all rooms across all resorts or filtered by resort.

Access: Staff, Manager, Admin only.

Query parameters:

Param	Type	Description
`resort_id`	UUID	Filter by resort
`status`	ENUM	Filter by status
`type`	ENUM	Filter by room type
`floor`	INT	Filter by floor

Response `200 OK`:


```
{
  "rooms": [
    {
      "id": "uuid",
      "resort_id": "uuid",
      "room_number": "201",
      "type": "deluxe",
      "floor": 2,
      "size_sqm": 42,
      "max_guests": 2,
      "bed_configuration": "1 King",
      "status": "available",
      "base_price_cents": 18000,
      "accessible": false,
      "amenities": ["Lake View", "WiFi", "Fireplace"],
      "updated_at": "2025-10-20T08:00:00Z"
    }
  ],
  "total": 150
}
```


`POST /api/rooms`

Creates a new room. Admin only.

Request body:


```
{
  "resort_id": "uuid",
  "room_number": "301",
  "type": "suite",
  "floor": 3,
  "size_sqm": 85,
  "max_guests": 3,
  "bed_configuration": "King + Sofa Bed",
  "base_price_cents": 42000,
  "accessible": false,
  "smoking": false,
  "amenities": ["uuid-lake-view", "uuid-jacuzzi", "uuid-wifi"]
}
```

Response `201 Created`:

```
{
  "id": "uuid",
  "room_number": "301",
  "status": "available",
  "message": "Room created successfully."
}
```

Error responses:

Code	Reason
`409`	Room number already exists for this resort
`422`	Invalid room type or missing required fields

`PATCH /api/rooms/:id/status`

Updates room status. Validates against state machine before applying.

Access: Staff (limited transitions), Manager, Admin (all transitions).

Request body:

```
{
  "status": "in_maintenance",
  "reason": "AC unit not functioning in room 201."
}
```

Response `200 OK`:

```
{
  "id": "uuid",
  "room_number": "201",
  "previous_status": "available",
  "new_status": "in_maintenance",
  "changed_at": "2025-10-20T09:15:00Z"
}
```

Error responses:

Code	Reason
`422`	Transition not allowed by state machine
`403`	Insufficient role for this transition

`GET /api/rooms/:id/status-log`

Returns full audit history of status changes for a room.

Access: Manager, Admin only.

Response `200 OK`:

```
{
  "room_id": "uuid",
  "room_number": "201",
  "log": [
    {
      "previous_status": "occupied",
      "new_status": "needs_cleaning",
      "reason": "Guest checked out",
      "changed_by": "system",
      "changed_at": "2025-10-19T11:45:00Z"
    },
    {
      "previous_status": "needs_cleaning",
      "new_status": "available",
      "reason": "Cleaning complete, room inspected",
      "changed_by": "staff-uuid",
      "changed_at": "2025-10-19T13:20:00Z"
    }
  ]
}
```

`GET /api/rooms/housekeeping/queue`

Returns rooms that need cleaning, ordered by priority.

Access: Staff, Manager, Admin.

Response `200 OK`:

```
{
  "queue": [
    {
      "room_id": "uuid",
      "room_number": "201",
      "resort": "Lakeside Resort",
      "floor": 2,
      "type": "deluxe",
      "status": "needs_cleaning",
      "priority": "high",
      "reason": "Next check-in in 2h 15m",
      "in_status_since": "2025-10-20T09:00:00Z"
    }
  ],
  "total": 8
}
```


11. Real-time Updates

Room status changes are broadcast in real-time via WebSockets to connected staff dashboards. This allows the front desk and housekeeping supervisors to see the live state of every room without polling.

Event	Broadcast to
Any room status change	Staff Dashboard (resort-scoped)
Room enters `needs_cleaning`	Housekeeping Dashboard
Room enters `in_maintenance`	Manager Dashboard + Alert
Room returns to `available`	Front Desk Dashboard

12. Business Rules Summary

- Room numbers must be unique per resort.
- A room cannot be set to `available` while an active reservation exists for current dates.
- Only Manager or Admin can override `in_maintenance` clearance.
- All status changes are logged permanently and cannot be deleted.

- A room must pass through `needs_cleaning` before returning to `available` after any occupied stay. Skipping this step is not permitted by the state machine.
- Rooms flagged `accessible = true` can only have their accessibility flag removed by Admin.
- Base price changes do not affect confirmed reservations (price is snapshotted at booking time).

13. Error Handling

Scenario	Response
Illegal state transition attempted	`422 Unprocessable Entity` with current and attempted status
Duplicate room number for resort	`409 Conflict`
Room set to maintenance with active reservation	`200 OK` + Manager alert raised
Amenity assigned to room that already has it	`409 Conflict`
Status log queried for non-existent room	`404 Not Found`

Service Module

Service Management Module — Detailed Design Document

1. Overview

The Service Management Module is designed to manage, sell, and track all non-accommodation offerings within the resort. This includes high-adrenaline activities, spa treatments, and rentals.

The system must support two primary workflows:

1. Hotel Guests: Services are added to a "Room Folio" for post-payment.
2. Day Visitors: Services are paid for upfront (Point of Sale) without a room association.

2. Responsibility (Roles)

3. Data Model

3.1 Core tables

Table: **services**

Column Name	Data Type	Description
id	UUID	Primary Key
name	VARCHAR	e.g., "Horse Riding", "Paintball Group"
category	ENUM	'Activity', 'Spa', 'Rental'
base_price	DECIMAL	Standard price per unit
unit_type	VARCHAR	'Rounds', 'KM', 'Arrows', 'Bullets', 'Minutes'
unit_quantity	INT	e.g., 50 (for bullets), 4 (for rounds)

Table: service images

Column Name	Data Type	Description

id	UUID	Primary Key
service_id	UUID	FK -> services
caption	VARCHAR	Images of the activity
sort_order	INT	Sorting order of the image
created_at	TIMESTAP	For tracking purpose

Table: **service_constraints**

Column Name	Data Type	Description
id	INT	Primary Key
service_id	UUID	Foreign Key → services.id
constraint_type	ENUM	'Min_Height', 'Max_Weight', 'Min_Age', 'Supervision_Req'
value	DECIMAL	e.g., 1.2 (meters), 100 (kg), 16 (years)

Table: *service_bookings*

The transaction record

Column Name	Data Type	Description
id	UUID	Primary Key
customer_id	UUID	Guest or walk-in customer
room_id	UUID	NULL if walk-in; FK → rooms.id

service_id	UUID	FK → services.id
booking_date	DATE	Service date
total_price	DECIMAL	Price × quantity
status_id	INT	FK → status table

3.2 Relationships

- **Services 1:N Constraints:** One service (Zipline) has multiple safety rules (Height and Weight).
- **Services 1:N Bookings:** One service definition can be sold thousands of times.
- **Customer 1:N Bookings:** A single customer can book multiple activities in one day.
- **Room 1:N Bookings (Optional):** A room folio aggregates multiple service bookings.

4. Status lifecycle

Tracking the state of a service is vital for safety and revenue.

1. **PENDING:** Booked but not yet paid (common for room guests).
2. **PAID:** Transaction complete (common for day users).
3. **CHECKED_IN:** Guest is at the activity site; instructor is verifying safety.
4. **IN_PROGRESS:** The guest is currently riding/getting a massage.
5. **COMPLETED:** Service finished; no further action needed.
6. **CANCELLED:** Guest did not show or safety check failed.

5. Other Details & Business Logic

5.1 The "Logic Engine" for Safety

When a service_booking is created, the system should trigger a **Safety Validation Hook**.

> **Example:** If the user selects "Zipline," the UI must prompt for "Height" and "Weight." If Weight > 100kg, the system prevents the booking.

5.2 Pricing Calculation

Prices are calculated using the following formula:

- Total = (Unit_Price × Number_of_Participants) + Tax

5.3 Inventory Management (Bicycles/Karts)

For services like "Pedal Kart" or "Bicycle," the system should track **Resource Availability**.

- If the resort only has 10 Bicycles, the system should not allow 11 "Bicycle" bookings for the same time slot.